

KAOS — Interview Question Bank (30 Questions + Model Answers)

Questions are grouped by theme. Each has a **crisp answer** you can say out loud, plus a **"go deeper"** note for follow-ups. Aimed at an **AI / AI-Engineering** interview, so the agentic and LLM questions are weighted heavily — but a good interviewer will probe systems design too.

A. System Design & Architecture

Q1. Give me the 60-second overview of KAOS.

KAOS is an event-driven, closed-loop CI/CD orchestration platform. External signals — Sentry errors, GitHub PR webhooks, deployment status — enter through an Ingestion service that normalizes them into clean Kafka events. Three autonomous agents consume those events by lifecycle phase: a **Triager** routes bugs to the right owner using an org graph and files Jira/Notion/Slack; a **Review Manager** assigns reviewers and shepherds the PR through approval and merge; an **Ops Manager** (an LLM agent) handles deployment, and on failure uses Bedrock to diagnose logs, notifies the people responsible, and **re-injects the failure as a new bug report** — closing the loop. A chatbot lets humans query the whole event history in natural language, and a Control Plane orchestrates and simulates everything.

Go deeper: The signature property is the closed loop — a prod failure becomes a pre-assigned bug and the lifecycle re-runs automatically.

Q2. Why three separate agents instead of one service?

Separation of concerns by lifecycle phase and trust boundary. Each agent has a different blast radius: the Review Manager is explicitly forbidden from touching production; the Ops Manager is the only thing that can. Splitting them means a bug in reviewer-routing can never trigger a deploy. It also maps cleanly to Kafka consumer groups — each agent is its own group with its own offset cursor, so they scale, restart, and fail independently.

Go deeper: It prevents "state leakage" — the intermediate state of a PR review never bleeds into deployment logic because they don't share a process or memory.

Q3. This is choreography, not orchestration. Explain the difference and why you chose it.

In **orchestration**, a central conductor calls each step in order (it knows all the participants). In **choreography**, each service reacts to events and emits new ones; nobody is "in charge." KAOS is choreographed: Ingestion doesn't know Triager exists — it just publishes a **QualityReport**. **Why:** decoupling. I can add a new consumer (say an analytics dashboard) to any topic without touching producers, and a slow agent can't block the others.

Go deeper: The cost is harder global reasoning — there's no single place that shows the whole flow, which is exactly why I built the Control Plane and the event store for observability.

Q4. Walk me through what happens when a deployment fails.

The CI/CD system POSTs to `/webhooks/deployment`; Ingestion publishes a `FAILED` event to `ops.deploy.status`. The Ops Manager (LLM agent) consumes it and: (1) calls `analyze_deployment_failure`, which sends logs to **AWS Bedrock (Titan)** for a root-cause + fix; (2) comments on Jira and sets Notion to "Needs attention"; (3) DMs **both** the author and the reviewer with the diagnosis; (4) calls `emit_quality_report`, publishing a **new** `QualityReport` to `system.quality.reports` with `suggested_assignee = author`; (5) logs a deployment report. Step 4 is the closed loop — Triager picks it up as a fresh, pre-assigned bug.

Q5. How would you scale this to 10,000 events/sec?

Kafka already gives horizontal scale via **partitions + consumer groups** — add partitions and run N replicas of each agent; each replica owns a subset of partitions. Bottlenecks would be the synchronous external API calls (Jira/Notion/Slack) and Bedrock; I'd make those async with a worker pool or move side-effects to their own topics (outbox pattern). I'd swap SQLite for Postgres (it's already abstracted behind `DATABASE_URL`), add `pgvector` for real semantic search, and put a rate-limiter/circuit-breaker around each external integration.

Go deeper: Partition key choice matters — keying by `service_name` keeps all events for one service ordered on one partition, which preserves the per-service state machine.

Q6. What are the failure modes and how does the system handle them?

Three layers: (1) **Transient API failures** → `retry_with_backoff` decorator with exponential backoff + jitter. (2) **Agent crashes** → Kafka offsets mean a restarted agent resumes from where it left off; nothing is lost. (3) **Duplicate processing** (from at-least-once delivery) → the SQLite idempotency check (`get_active_jira_ticket`) prevents duplicate tickets. Malformed events are caught by `JSONDecodeError` handling in the consumer base and skipped rather than crashing the loop.

B. Kafka & Event-Driven Messaging

Q7. Why Kafka specifically, over RabbitMQ or a REST mesh?

Kafka gives me **durable, replayable logs** with **consumer-group fan-out**, which a traditional message queue (delete-on-consume) doesn't. I need replay (restart an agent and reprocess), multiple independent consumers per topic, and a built-in audit trail. The topic is the history of what the org experienced. A REST mesh would tightly couple services and lose buffering/back-pressure.

Q8. Explain your topic taxonomy. Why split `dev.pr.updates` from `dev.pr.decisions` ?

Topics are named by **semantic intent**, not source system. `updates` are high-frequency mechanical events (every push/sync). `decisions` are low-frequency, high-importance human signals (approve/reject). Separating them lets me tune retention and partitioning independently and keeps each consumer's branching logic simpler. It also means I could later route decisions to a stricter, audited consumer without touching the noisy update stream.

Q9. What delivery guarantee do you have, and is it correct for this use case?

Currently **at-least-once** — I rely on Kafka's default auto-commit, so a crash after side-effects but before commit means re-processing. For a system that *creates tickets and sends DMs*, that's risky, which is why I added application-level idempotency (the recurring-ticket check). To get closer to effectively-once I'd switch to **manual offset commits after successful processing** and make the side-effects idempotent (e.g., dedup keys on Jira/Notion creation).

Go deeper: True exactly-once across external systems is impossible without those systems supporting idempotency keys; the honest target is at-least-once + idempotent consumers.

Q10. How does the consumer base class work and what pattern is it?

It's the **Template Method pattern** (`shared/kafka/consumer_base.py`). The base owns the invariant infrastructure — subscribe, poll loop, decode, error handling, partition assign/revoke callbacks, graceful shutdown — and exposes one abstract `process_message()` hook. Each agent subclasses it and implements only its business logic. This removed ~40 lines of duplicated polling per agent and guarantees uniform error handling.

C. Neo4j & the Knowledge Graph

Q11. Why a graph database instead of Postgres for the org data?

Because the core questions are **relationship traversals**: "Who owns this service → if they're on leave, who worked on it → if nobody, who do they report to?" In Cypher that's a one-liner; in SQL it's recursive joins. The data is naturally a graph (people, services, ownership, reporting lines), so I modeled it as one.

Q12. Explain the "active routing" algorithm.

Three-tier fallback in `find_service_owner` : (1) direct `OWNS` owner who is `status='active'` ; (2) else any active `WORKED_ON` contributor; (3) else escalate up `REPORTS_TO` . The key trick is filtering `status='active'` **inside Cypher**, so an on-leave owner is never returned — routing is correct by construction, not by post-filtering in Python. The reviewer query is the mirror: Senior + Active + owns the service + **not** the PR author (prevents self-review, load-balances to qualified people).

Q13. You normalize service names in the query. Why, and what's the risk?

External systems are inconsistent — `payment-service` , `PaymentService` , `payment_service` all refer to the same thing. I normalize with `toLowerCase(replace(replace(name, '-', ''), '_ ', ''))` on both sides so

matching is tolerant. **Risk:** two genuinely different services that collapse to the same normalized string would collide; in production I'd use a canonical service ID rather than fuzzy name-matching.

Q14. Your Neo4j client is just one `query()` method. Defend that.

The intelligence belongs in the Cypher, which lives next to the business logic that owns it (each agent's `tools.py`), not buried in a fat repository layer. A thin client keeps queries readable and co-located with intent. The trade-off is no compile-time safety on queries — acceptable for this scale; at larger scale I'd add a typed query layer or OGM.

D. Agents, LLMs & the Agentic Core (the AI-heavy section)

Q15. Why did you hand-roll the ReAct loop instead of using LangChain's AgentExecutor?

Three reasons: **transparency** (every reasoning/tool step is visible and debuggable — no framework magic), **a minimal dependency surface** (I'm not pinned to a fast-moving `AgentExecutor` API), and **guardrails** (an explicit 5-iteration cap and `temperature=0` give bounded, deterministic behavior). When every tool call has a real side effect — creating tickets, DMing people — I want full control over the loop, not a black box.

Go deeper: The loop is literally: invoke LLM → if `tool_calls`, execute each and append `ToolMessage` → else return content → cap at 5 iterations. It still uses LangChain's `ChatBedrock` and `@tool` schema binding; I just own the control flow.

Q16. When do you use a deterministic pipeline vs an LLM agent? This is a key design decision.

I split by predictability. **Triager and Review Manager are deterministic** — they're well-defined state machines (find owner → Jira → Notion → Slack, in that exact order). I don't want an LLM creatively skipping the Jira step. **Ops Manager and Chatbot are LLM agents** — failure diagnosis and free-text Q&A are open-ended, so I *want* the model to choose tools. The principle: **use the LLM for judgement, use code for procedure**. Reliability where the path is known; flexibility where it isn't.

Go deeper: The repo even keeps an LLM `main.py` and a deterministic `consumer.py` for Triager — the story is "prototyped with an agent loop, then hardened the predictable path into deterministic code." That's a real maturity arc.

Q17. Why `temperature=0` and a 5-iteration cap?

`temperature=0` because routing and ops decisions should be **reproducible** — the same bug should route the same way every time; creativity is a liability here. The **5-iteration cap** bounds cost and latency and prevents an infinite tool-calling loop (a model that keeps re-calling a tool would otherwise run forever and rack up side-effects). They're cheap, blunt safety rails appropriate for a system with real-world actions.

Q18. How do tools work in your agent, and how does the model know what's available?

Each tool is a Python function decorated with LangChain's `@tool`; the decorator turns the docstring + type hints into a JSON schema. `llm.bind_tools(tools)` ships those schemas to Bedrock so the model can emit structured `tool_calls`. My loop dispatches each call through a `tool_map` (name → function), runs it, and feeds the result back as a `ToolMessage`. So the *docstrings are prompt engineering* — they're how the model decides which tool fits the task.

Q19. The Chatbot is essentially RAG. Explain.

Yes — it's **retrieval-augmented generation over ops history**. The "documents" are events in the SQLite store and nodes in Neo4j. The model retrieves via tools (`get_bug_timeline`, `search_events`, `find_team_info`, `get_jira_status`, `get_bug_solution_details`) rather than a vector DB, then synthesizes a natural-language answer. `get_bug_timeline` is the nicest part: it deterministically reconstructs a readable narrative (🐛 → 📄 → 👤 → ✅ → 🚀) which the LLM then summarizes — grounding the answer in real events so it can't hallucinate the timeline.

Go deeper: I store sentence-transformer embeddings per event for future semantic retrieval, but today retrieval is keyword/graph-based. The honest next step is `pgvector` similarity search.

Q20. You use two different Bedrock models. Why?

`BaseAgent` uses **Amazon Nova Lite** (`amazon.nova-lite-v1:0`) for the agentic loop — it's cheap, fast, and supports tool-calling, which is all I need for routing-style reasoning. The Ops Manager's `analyze_deployment_failure` tool uses **Amazon Titan Text Express** for free-form log summarization. The principle is **right-sizing the model to the task** — I don't pay for a frontier model to decide which of four tools to call.

Q21. How do you prevent the agent from doing something harmful or looping forever?

Layered guardrails: (1) **bounded autonomy** — 5-iteration hard cap; (2) **constrained toolset** — the agent can only call the specific tools I bind, so its action space is finite and known; (3) **deterministic decoding** (`temp=0`); (4) **explicit system-prompt rules** — e.g., the Triager prompt says "call each tool exactly once; if a duplicate is found, STOP." (5) For the truly critical paths I bypass the LLM entirely and run deterministic code. The agent can't invent an API call it doesn't have a tool for.

Q22. How would you evaluate or test these agents?

For deterministic agents, ordinary unit/integration tests with mocked tools (the repo has `tests/test_escalation.py` and connectivity checks). For LLM agents, I'd build an **eval set** of input events with expected tool-call sequences and assert the trajectory (did it call `analyze` → `notify` → `emit_quality_report`?), plus golden-output checks on the diagnosis quality. I'd also add **tool-argument validation** and log every trajectory for offline review. Right now testing leans on the Control Plane's simulation endpoints for end-to-end smoke tests.

E. State, Memory & Data

Q23. You have Kafka, Jira, Notion — why *also* a SQLite store?

Three jobs Kafka/SaaS can't do cheaply: (1) **Idempotency** — "is there already an active ticket for this service?" is one indexed query, and it's what stops the closed loop from spamming duplicate tickets. (2) **Fast cross-system linkage** — service → live Notion page-id / Jira key without hitting their APIs every time. (3) **Queryable history** — Kafka is a stream, not a query engine; the chatbot needs `SELECT`-able events. It's abstracted behind `DATABASE_URL`, so it's a one-line swap to Postgres.

Q24. Tell me about the embeddings. Are they actually used?

`log_event` lazily loads `all-MiniLM-L6-v2` (sentence-transformers) and stores a vector per event for future semantic search ("have we seen an NPE like this before?"). **Honest answer:** the vectors are *persisted but not yet queried* — `search_events` currently uses SQL `LIKE`. A production version moves to Postgres + `pgvector` and does cosine-similarity retrieval. I keep the embedding pipeline in place so the data is ready when the search layer lands.

Go deeper: Being upfront that "stored embeddings ≠ a working semantic index" signals you understand the difference, which interviewers probe for.

Q25. How do you guarantee a service doesn't get two tickets for the same recurring bug?

The Triager calls `get_active_jira_ticket(service)` before creating anything. If an active ticket exists, it **updates** it ("⚠️ Recurring Incident") and returns early instead of creating a new one. This is essential because the Ops Manager's closed loop can re-emit the same failure repeatedly — without the guard, every deploy retry would mint a new ticket.

F. Integrations & Edge

Q26. Why a dedicated Ingestion service instead of agents consuming webhooks directly?

It's an **anti-corruption layer** (DDD). External payloads (Sentry, GitHub) are messy and change without notice. Ingestion is the single place that understands their wire format and translates to clean, Pydantic-validated internal events. If GitHub changes a field, I patch one file, not five agents. It also **acks webhooks immediately** (FastAPI `BackgroundTasks` publish to Kafka *after* the 200) because webhook senders time out fast and retry aggressively.

Q27. How does the Control Plane manage agent processes, and what are the risks?

`process_manager.py` launches each agent as a child `subprocess.Popen`, tracks it by PID, and stops it via `psutil` (terminating child processes too). It injects `PYTHONPATH` and `PYTHONUNBUFFERED=1` for live logs. **Risks:** it's a single-host, single-worker design — fine for a demo, but not HA. In production these

would be containers under Kubernetes/ECS with health checks and restart policies, not a Python subprocess manager.

Q28. How are secrets and config handled?

`config/settings.py` uses Pydantic-Settings `BaseSettings` — typed fields loaded from `.env`, never hard-coded. Optional integrations are typed `str | None` so the system boots even if, say, GitHub isn't configured (the chatbot degrades to "simulation mode"). In production I'd back this with AWS Secrets Manager / SSM rather than a `.env` file.

G. Reflection & Trade-offs (behavioral — they *will* ask these)

Q29. What's the weakest part of this system and how would you fix it?

The honest weak points are demo simplifications: merge-conflict detection is stubbed (`check_pr_status` always returns CLEAN), Bedrock analyzes hard-coded sample logs instead of the real `logs_url`, embeddings aren't searched yet, and delivery is at-least-once with auto-commit. My priority fixes: (1) wire `analyze_deployment_failure` to fetch the real logs from the `logs_url` / CloudWatch; (2) implement true merge-conflict polling against the GitHub API; (3) move to manual offset commits + idempotency keys for stronger delivery semantics; (4) stand up `pgvector` for real semantic incident search.

Go deeper: I'd rather name these than have them found — each one has a clear, scoped fix, which shows I understand the gap between a portfolio build and production.

Q30. If you rebuilt KAOS from scratch, what would you change?

Four things: (1) **One execution model** — pick deterministic-with-LLM-tools consistently instead of having both `main.py` and `consumer.py` variants. (2) **A real outbox pattern** — write side-effects to an outbox topic and have dedicated, idempotent workers perform Jira/Notion/Slack actions, so the agents stay pure decision-makers. (3) **Containerized deployment** with proper health checks instead of the subprocess Control Plane. (4) **An eval harness for the agents** from day one — trajectory tests and golden diagnoses — because LLM behavior regresses silently without it. The core event-driven, graph-routed, closed-loop architecture I'd keep — that part is sound.

Quick-Fire Round (rapid recall)

#	Question	One-line answer
31	What model powers the agent loop?	Amazon Nova Lite via Bedrock (<code>ChatBedrock</code>), temp 0
32	What closes the loop?	<code>emit_quality_report</code> re-publishes failures to <code>system.quality.reports</code>
33	Where does "who owns this?" come from?	Neo4j <code>OWNS</code> / <code>WORKED_ON</code> / <code>REPORTS_TO</code> traversal
34	When does Jira go to "Done"?	Only on prod deploy success , owned by Ops Manager — not on merge
35	What guarantees no duplicate tickets?	SQLite <code>get_active_jira_ticket</code> idempotency check
36	Which design pattern is the consumer base?	Template Method
37	Which service is the anti-corruption layer?	Ingestion
38	How many Kafka topics, and why?	Four, named by semantic intent (quality / pr.updates / pr.decisions / deploy)
39	How are tools described to the LLM?	<code>@tool</code> docstrings + type hints → JSON schema via <code>bind_tools</code>
40	Why temperature 0?	Reproducible routing/ops decisions; creativity is a liability here

